UNIVERSITY OF RWANDA

GAKO CAMPUS

COLLEGE OF SCIENCE AND TECHNOLOGY
COMPUTER ENGINEERING

REAL-TIME SYSTEMS

LABORATORY 2 REPORT

Mini Project 1.2:
Measurement-Based WCET Estimation and Confidence Tool

Programme	Computer Engineering
Level	4
Group	6
Date	30 September 2026
Registration Numbers	223023475, 223026683, 223027315

Submitted for the Real-Time Systems course

Github link: https://github.com/elyse121/group6--RTS/tree/main/Lab2_WCET

Table of Contents

A dynamic table of contents can be updated in Microsoft Word by right-clicking the table and selecting "Update Field".

Table of Contents

1. Title

Measurement-Based Worst-Case Execution Time (WCET) Estimation: A Reusable Tool, Random versus Adversarial Inputs, and the Limits of Measurement.

2. Objective

The objective of this mini project was to develop a reusable Python tool for estimating the Worst-Case Execution Time (WCET) of a target function using a measurement-based approach. The tool also evaluates statistical behaviour and the inherent unsoundness of measurement-based estimation, and compares representative random inputs with deliberately constructed adversarial inputs that approximate the worst-case execution path.

3. Background

Hard real-time schedulability analysis requires an upper bound C on the execution time of each task. Section 1.7 of the course material describes two main approaches. Static analysis reasons about possible execution paths and the hardware model. It aims to be sound, meaning that the derived bound should not underestimate the true WCET, although it can be pessimistic. Measurement-based estimation instead executes the program and records observed execution times. It is comparatively inexpensive and measures behaviour on real hardware, but it is unsound because a finite test campaign cannot guarantee that the true worst-case input, execution path, or hardware state has been observed.

Therefore, a measured maximum is an observation rather than a proof of WCET. Average execution time is also not sufficient for hard real-time guarantees. For this reason, the tool reports tail statistics including P90, P95, P99 and the maximum. A configurable safety margin is also included, following the 60–70% utilization-budget convention discussed in the course material.

4. Tools Used

- Python 3.12
- time.perf_counter() for individual execution-time measurements
- NumPy 2.4 for arrays, percentiles and random input generation
- Matplotlib 3.10 for histogram visualization
- Python standard-library modules including gc, csv, argparse and dataclasses
- Execution platform: a single-core Linux sandbox (shared, non-real-time), which is important when interpreting timing noise.

5. Methodology

1. Three target functions with different control-flow structures were implemented.
2. Two input strategies were prepared for each function: random/representative inputs and deliberately adversarial inputs.
3. A generic measurement harness timed every function call individually using time.perf_counter(), while retaining every measured sample.
4. A statistics module calculated the minimum, mean, maximum, standard deviation, P90, P95, P99, outlier indicators and a margined WCET estimate.
5. Histograms and CSV/Markdown result tables were generated for analysis.
6. The measurement campaign was repeated 10 times to examine the stability of the observed maximum and the P99 percentile.

6. Target Functions

No.	Function	Control-flow shape	Inputs associated with longer execution
1	early_exit_search(key)	Linear scan of a 2000-element list with early return	Keys near the end or absent from the list; an absent key forces all 2000 iterations.
2	collatz_steps(n)	Data-dependent loop	Values producing long Collatz sequences; the worst n in [1, 100000] is 77031 with 350 steps.
3	classify_packet(pkt)	Three-level nested conditionals	The deep error-recovery path: priority > 5, flag bit 0 set and invalid checksum, producing 300 inner iterations.

The functions were intentionally different in structure. The measurement harness treats each target as a black box and does not depend on its internal implementation.

7. Random Input Strategy

- early_exit_search: the key was always present and selected at a uniformly random position. The actual mean position was 979 and the maximum position was 1991 out of 2000.
- collatz_steps: n was selected uniformly from 2 to 100000. The actual mean sequence length was 110 steps and the longest random input produced 306 steps, compared with the known worst case of 350 steps.
- classify_packet: packet size was randomized from 0 to 1500, priority from 0 to 9, flags from 0 to 3, and checksum validity was 90%. The worst path has an approximate probability of 2%; in the 500-call campaign it was reached 10 times.

8. Adversarial Input Strategy

- early_exit_search: keys absent from the data were selected so that the early return never occurs and all 2000 iterations execute.
- collatz_steps: the exhaustively identified worst input, n = 77031, was used, producing 350 Collatz steps.
- classify_packet: inputs were deliberately selected with priority > 5, flag bit 0 set and an invalid checksum, forcing the 300-iteration recovery path.

These inputs are considered hand-derived because the longest logical path was identified by reasoning about the source code rather than relying only on random sampling.

9. Measurement Procedure

The generic measurement function was implemented as measurement.measure(func, inputs, repeats=500, warmup=20). The procedure was:

7. Generate or accept all inputs outside the timed region.
8. Execute 20 warm-up calls without timing them.
9. For each of the 500 inputs, record the elapsed time using t0 = perf_counter(); func(x); t1 = perf_counter().
10. Pause garbage collection during timing so that a garbage-collection pass is not incorrectly attributed to the target function.
11. Return a Measurement object containing all 500 durations and the corresponding inputs.

The measured timer overhead of an empty start/stop pair was approximately 40–50 ns at the median. This overhead was not subtracted because doing so would make the estimate less conservative. Across six experiments, 3000 timed executions were recorded in results/raw_samples.csv.

10. Statistical Results

All execution times are reported in microseconds (µs), using 500 samples per experiment. The recommended WCET is calculated as Max × 1.5.

Function	Strategy	Min	Mean	Max	Std Dev	P90	P95	P99	Recommended WCET
early_exit_search	random	0.27	28.84	124.47	20.46	52.13	64.15	91.75	186.71
early_exit_search	adversarial	53.50	63.25	163.20	17.11	88.77	101.46	133.53	244.79
collatz_loop	random	1.35	7.10	46.21	3.95	11.47	12.90	20.25	69.31
collatz_loop	adversarial	21.95	25.18	205.91	12.75	26.65	36.90	76.34	308.87
nested_packet	random	0.71	3.65	68.92	4.71	8.31	9.07	19.65	103.38
nested_packet	adversarial	16.14	17.90	76.07	5.40	18.52	27.73	36.31	114.11

10.1 Outlier Analysis

The tool flags a campaign when Max/P99 > 1.5, indicating that the maximum is substantially above the 99th percentile. It also counts extreme observations above Q3 + 3×IQR using Tukey's fence.

Function	Strategy	Max/P99	Outlier flag	Extreme outliers
early_exit_search	random	1.36	No	0
early_exit_search	adversarial	1.22	No	79
collatz_loop	random	2.28	YES	3
collatz_loop	adversarial	2.70	YES	69
nested_packet	random	3.51	YES	13
nested_packet	adversarial	2.09	YES	60

The means of the adversarial runs were approximately 2.2×, 3.5× and 4.9× the corresponding random means, showing that the adversarial inputs exercised longer logical paths. However, the maxima were less orderly. For example, collatz_loop had an adversarial maximum of 205.91 µs, which was 2.7× its P99 and about eight times its mean. Because the target function performs the same logical work for the repeated adversarial input, isolated spikes are consistent with operating-system scheduling, interrupts, CPU-frequency effects, cache behaviour and interpreter effects. The large Tukey counts in some adversarial rows also arise partly because their normal distributions are very tight, making the IQR small.

11. WCET Estimation

The tool calculates the recommended WCET using:

Recommended WCET = measured maximum × (1 + margin/100)

The default margin is 50%. A 50% margin means that the measured maximum occupies approximately 66.7% of the resulting budget, which lies within the 60–70% utilization convention discussed in Section 1.7.

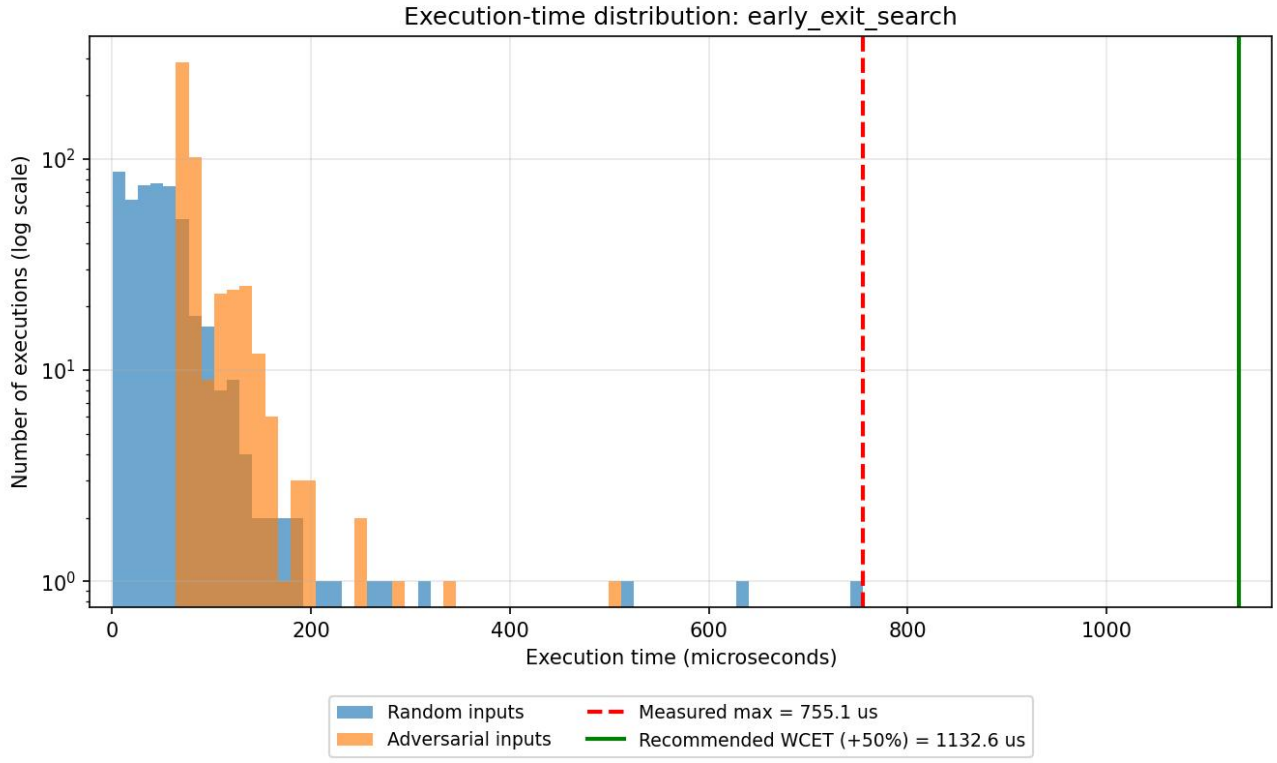
Function	Random Maximum (µs)	Adversarial Maximum (µs)	Adversarial/Random	Margined recommendation (µs)
early_exit_search	124.47	163.20	1.31	244.79
collatz_loop	46.21	205.91	4.46	308.87
nested_packet	68.92	76.07	1.10	114.11

Important: The measured maximum is not a guaranteed WCET. It is only the largest execution time observed during the tested executions. The true WCET may be larger. The safety margin is a heuristic cushion and does not convert an observation into a proof.

12. Graphical Results

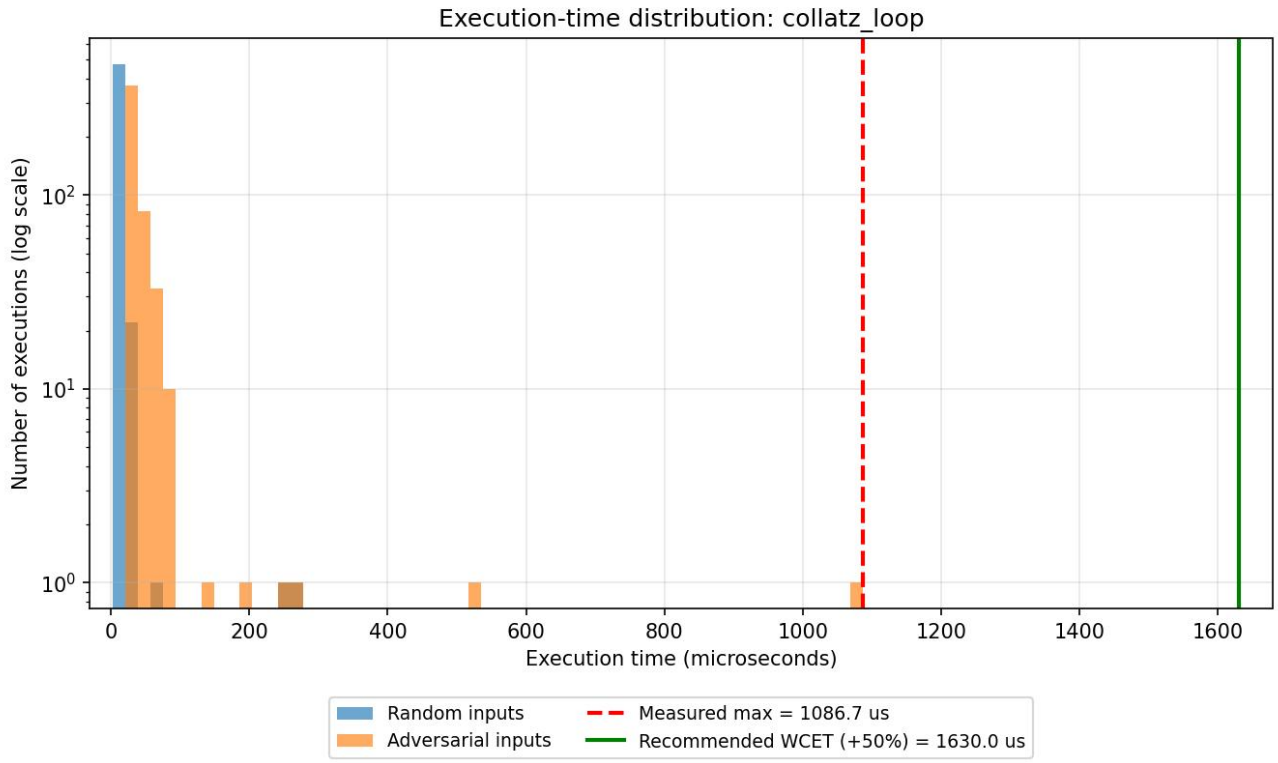
One histogram was produced for each target function. The random and adversarial distributions are overlaid using shared bins. Counts are shown on a logarithmic scale so that rare tail observations remain visible. The measured maximum and the margined recommendation are marked on each figure.

Figure: Early-Exit Search execution-time distribution



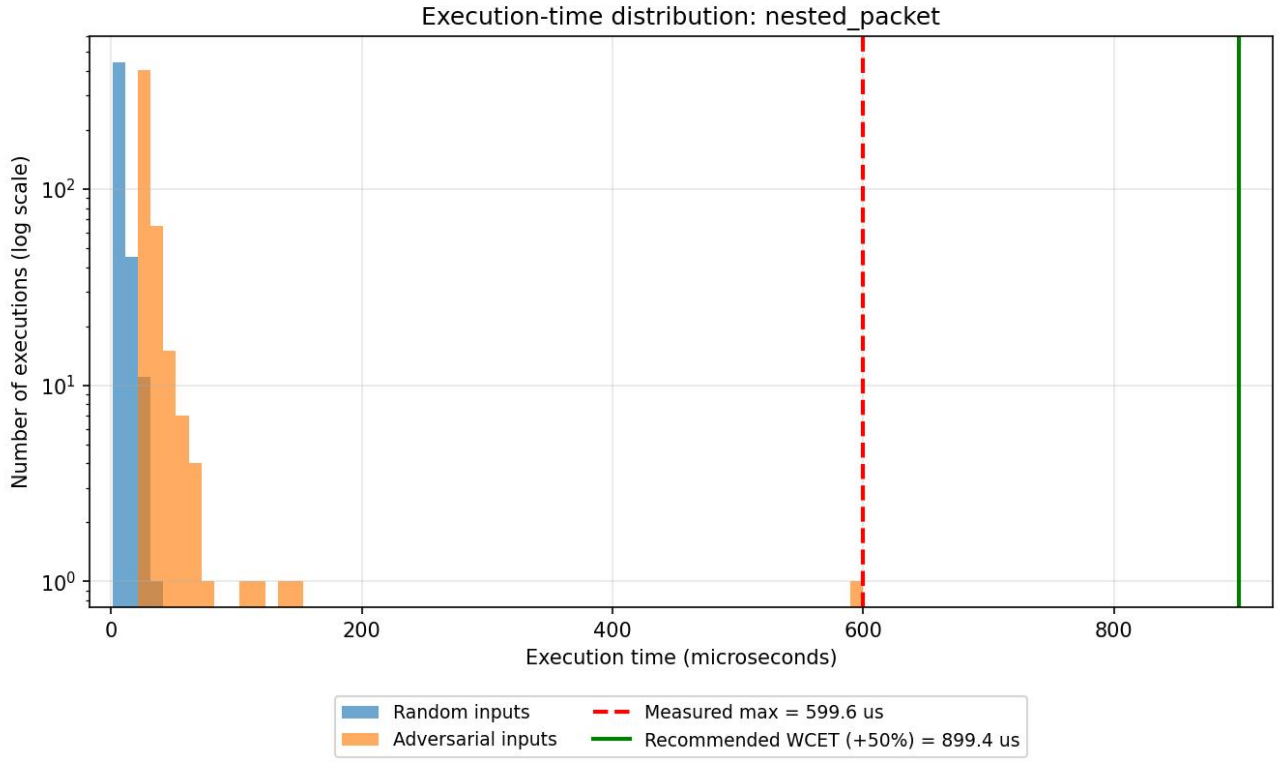
Measured maximum is an observation, NOT a guaranteed WCET.

Figure: Collatz Loop execution-time distribution



Measured maximum is an observation, NOT a guaranteed WCET.

Figure: Nested Packet execution-time distribution



Measured maximum is an observation, NOT a guaranteed WCET.

The main body of the adversarial distributions lies to the right of the corresponding random distributions, indicating longer logical paths. Each distribution also contains a thin right tail of isolated observations, which can strongly affect the measured maximum and consequently the margined estimate.

13. Random versus Adversarial Comparison

Function	Random Maximum (µs)	Adversarial Maximum (µs)	Adv/Random	Adversarial gave largest observed value?
early_exit_search	124.47	163.20	1.31	Yes
collatz_loop	46.21	205.91	4.46	Yes
nested_packet	68.92	76.07	1.10	Yes

In this campaign, the adversarial strategy produced the largest observed duration for all three functions. For nested_packet, the random strategy also reached the worst logical path 10 times out of 500 calls. Consequently, the distinction between the strategies is clearer in the mean and P99 than in the maximum, because the random maximum can be inflated by system noise. For collatz_loop, the random strategy did not reach n = 77031; its longest sequence was 306 steps, whereas the adversarial input produced the known 350-step sequence.

13.1 Repeatability Test

The same 500-sample experiment was repeated 10 times to examine the stability of the measured maximum and P99.

Function	Strategy	Max: min	Max: median	Max: largest	Max spread	P99 spread
early_exit_search	random	102.51	157.16	320.24	3.12×	2.13×
early_exit_search	adversarial	167.68	378.97	2132.46	12.72×	2.11×
collatz_loop	random	52.82	119.26	1050.88	19.89×	3.29×
collatz_loop	adversarial	72.58	144.89	1241.46	17.10×	2.96×
nested_packet	random	26.04	48.87	525.65	20.18×	3.23×
nested_packet	adversarial	74.13	132.58	1646.03	22.20×	1.80×

Although the program, inputs and seed were kept the same, the measured maximum changed substantially between campaigns. For example, the maximum for nested_packet under random inputs ranged from 26.04 µs to 525.65 µs. The P99 was considerably more stable than the maximum, although it also varied. This demonstrates why a measured maximum from one campaign cannot be treated as a guaranteed WCET.

14. Discussion

14.1 Functions with More Predictable Logical Worst Cases

early_exit_search has a simple and regular control-flow structure. Its logical worst case is the execution of all 2000 iterations, and the adversarial strategy reaches this path on every call. The random strategy also produced a key position of 1991, close to the end of the list. Therefore, the logical path coverage for this function is relatively straightforward, while the remaining uncertainty comes mainly from platform timing noise.

nested_packet has a similarly identifiable deep path. Both random and adversarial inputs can reach it, although the random strategy reaches it less consistently. The distinction is therefore more apparent in the distribution and percentile statistics than in a single maximum.

14.2 Function Most Vulnerable to Underestimation

collatz_loop illustrates a major weakness of random measurement. Its execution time depends on a mathematical property of the input rather than simply on an obvious loop bound. The random strategy did not discover the known worst input n = 77031, so relying only on random tests could underestimate the logical worst case. This demonstrates why measurement-based testing can miss rare paths or rare inputs.

14.3 Hardware and Operating-System Noise

The timing maxima were often much larger than the corresponding means and varied substantially across repeated campaigns. Possible sources include the Python interpreter, operating-system scheduling, interrupts, CPU-frequency changes, cache effects and other activity on the shared machine. Consequently, a measurement campaign on a general-purpose operating system cannot cleanly separate the target function's execution cost from all external timing interference.

14.4 Percentiles versus the Maximum

P99 is generally more stable than the maximum, but it is not a WCET bound. By definition, approximately one percent of observations can lie above P99. The Max/P99 indicator is therefore useful as a warning that the observed tail contains isolated spikes and that the maximum may be especially sensitive to measurement noise.

15. Limitations

12. Only tested executions are observed. The true worst-case input or hardware state may never be exercised.
13. Random testing may miss a rare worst-case path or reach it too infrequently for reliable characterization.
14. Adversarial inputs are hand-derived and depend on the tester's understanding of the target code. They do not capture all hardware effects.
15. The Python interpreter and general-purpose operating system introduce timing noise through scheduling, interrupts, cache effects and frequency scaling.
16. The safety margin is a convention-based heuristic rather than a statistical guarantee. A fixed margin cannot guarantee coverage of all possible timing variation.
17. The tool uses empirical statistics and does not fit an extreme-value statistical model to produce a probabilistic tail bound.
18. The results were obtained from interpreted code on a general-purpose platform and therefore should not be assumed to represent timing on a specific embedded processor or RTOS.

16. Soundness versus Pessimism

Section 1.7 presents a fundamental trade-off between soundness and pessimism. Static analysis seeks a sound upper bound by considering possible execution paths and relevant hardware states, but such bounds may be pessimistic. Measurement-based estimation works with observed executions and therefore avoids some of the conservatism associated with purely static reasoning, but it is unsound because untested paths and hardware states may produce longer execution times.

The repeatability results illustrate this limitation directly: the measured maximum changed substantially between otherwise identical campaigns, while the random strategy did not discover the known worst input of the Collatz function. Adding a 50% margin increases the budget and therefore adds practical conservatism, but it does not establish a mathematically or certification-level safe bound.

16.1 Why the Tool Alone Is Not Sufficient for Certification

A certification-oriented WCET argument requires evidence that the bound is safe over the relevant input space, execution paths and hardware behaviour. This measurement tool does not provide such a proof because it observes only a finite sample, operates on a non-real-time general-purpose environment, and uses a heuristic

safety margin. Its results can nevertheless be useful as supporting evidence, for example for early budgeting, comparison with static analysis, or soft real-time engineering decisions.

17. Conclusion

A reusable measurement harness, statistical analysis module and visualization process were implemented for three target functions with different control-flow structures. Each function was tested using both representative random inputs and deliberately constructed adversarial inputs. The results showed that adversarial inputs generally exercised longer logical paths and produced higher average execution times. At the same time, repeated campaigns demonstrated substantial variation in the measured maximum, while the random strategy failed to discover the known worst input of the data-dependent Collatz function.

The experiment therefore demonstrates the main lesson of measurement-based WCET estimation: a measured maximum is useful evidence about observed behaviour, but it is not a guaranteed WCET. A safety margin can provide additional engineering budget, but it cannot transform finite observations into a proof of safety. For systems requiring strong timing guarantees, measurement should be treated as complementary evidence alongside sound analysis methods rather than as the sole basis for certification.

18. Reproducibility

The project can be reproduced using the following commands from the Lab2_WCET project directory:

```
pip install -r requirements.txt
python main.py
python stability_check.py
```

Absolute timing values can differ across machines and across runs. The main qualitative findings of the experiment are the relative difference between representative and adversarial inputs, the instability of isolated maximum observations, and the greater—but still imperfect—stability of tail percentiles such as P99.